



Gradient Descent I

Prof. Gilad Gressel
Amrita School of Engineering

Introduction

- Calculate the MSE manually on a tiny dataset
- Draw intuitions for how we can use the gradient to help us

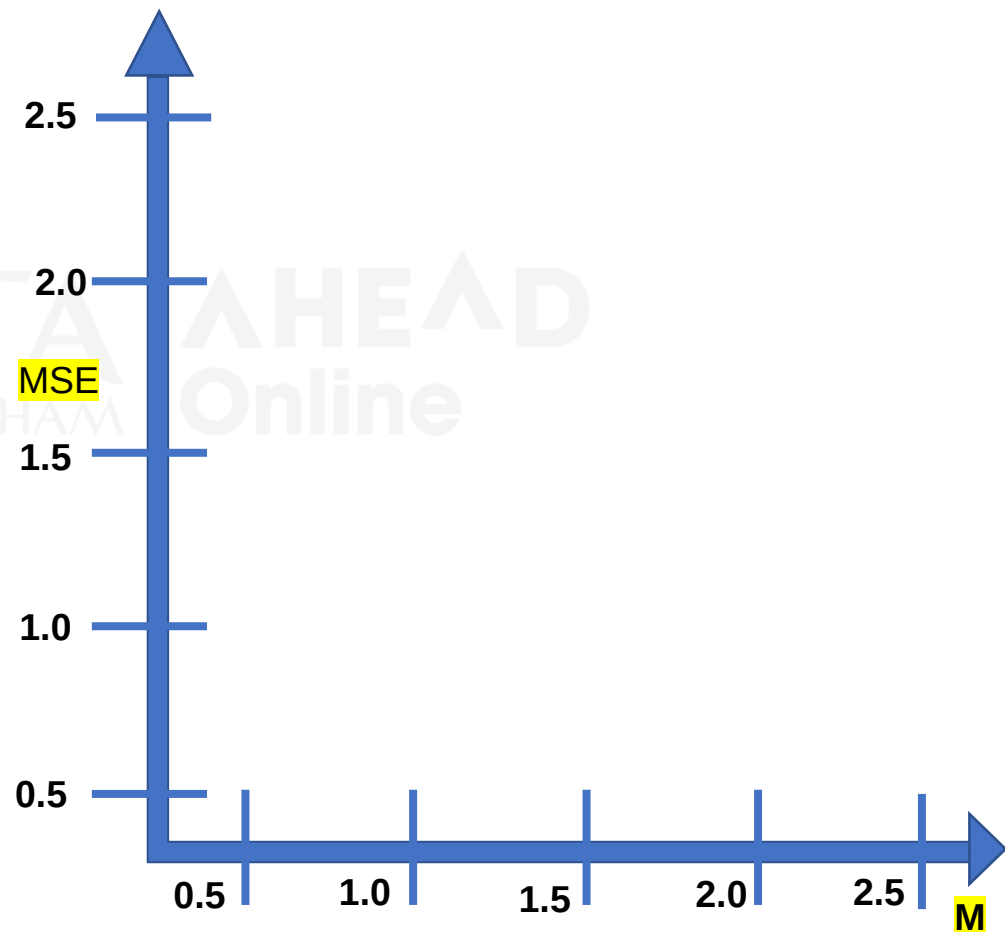
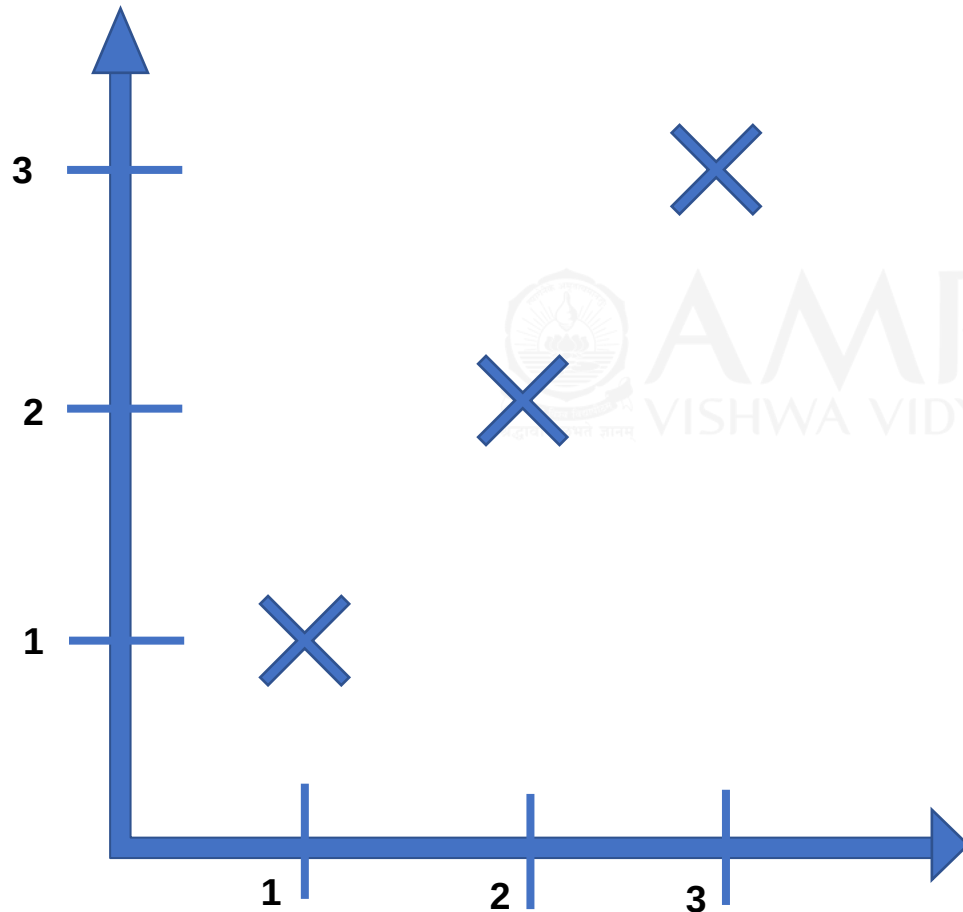
Place your
Webcam Video here
Size 100%

Computing the Cost Function

- Our function: $y = mx + c$
- $M = ?$, $c = 0$

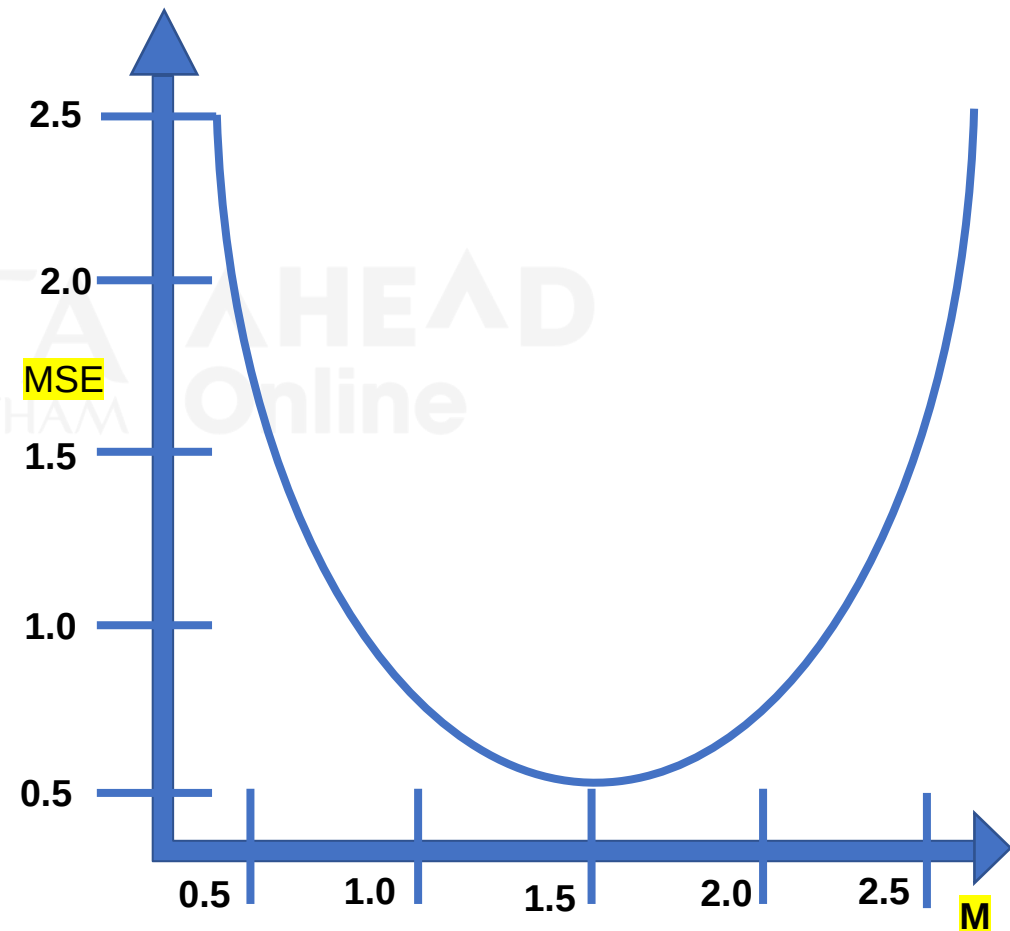
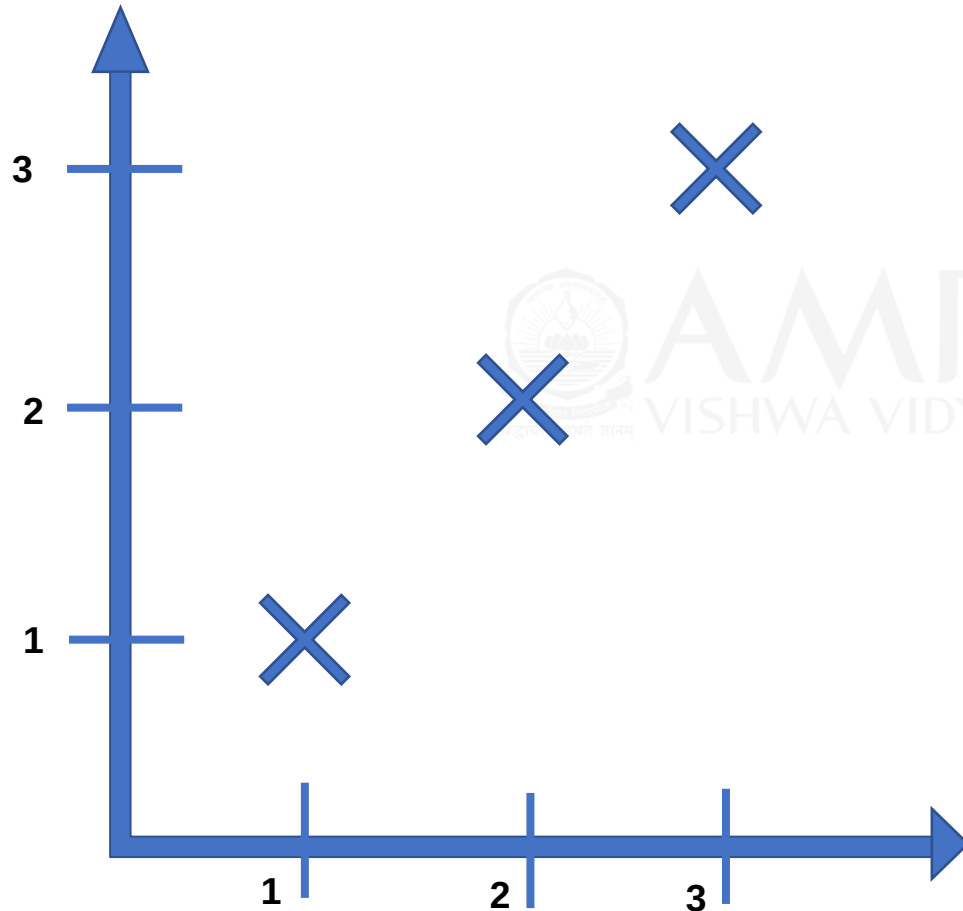
- Mean squared error

$$= \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2$$



Leveraging the Cost Function

- Given any guess of m (and c), what should we do after we evaluate our fit with the cost function?
- Using the gradient of the cost function, we can update m to reduce the MSE



Conclusion

- We manually calculated the MSE for a bunch of parameters
- We saw that MSE is convex
- Next we will use the gradient descent algorithm to automatically update m and c , our parameters

Place your
Webcam Video here
Size 100%



Gradient Descent II

Prof. Gilad Gressel
Amrita School of Engineering

Introduction

- Review the relationship between function and cost function
- The gradient descent algorithm in math
- Gradient descent in code
- Gradient of MSE

Place your
Webcam Video here
Size 100%

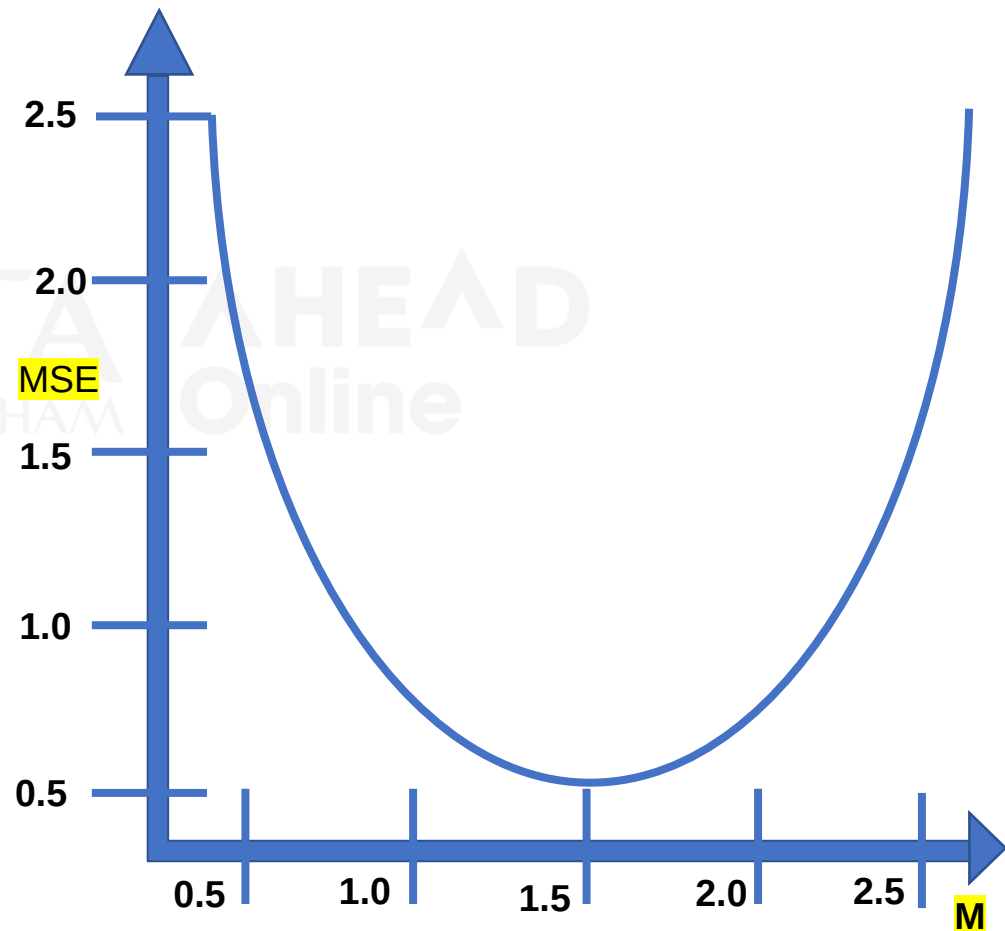
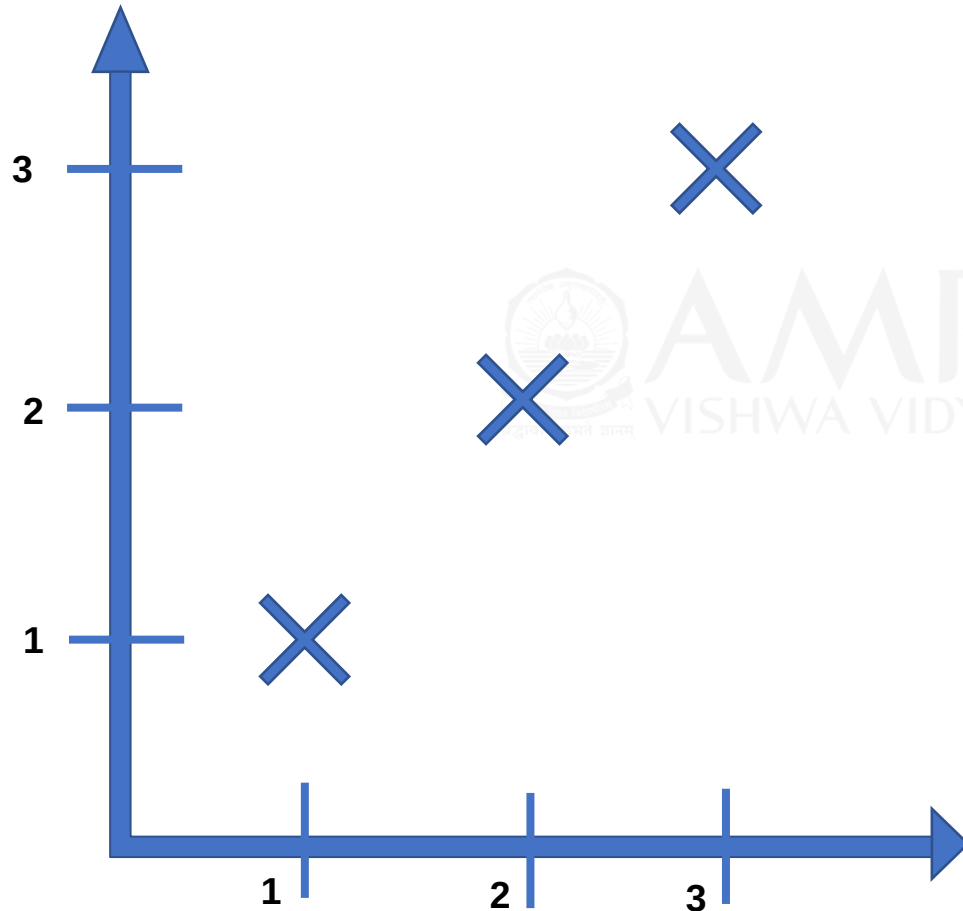
Gradient Descent Intuitions

- We want to use the gradient of the cost function
- We know that stepping down the gradient (towards the bottom) will improve our score
- Let's refresh our minds



Leveraging the Cost Function

- Given any guess of m (and c), what should we do after we evaluate our fit with the cost function?
- Using the gradient of the cost function, we can update m to reduce the MSE



Gradient Descent Algorithm

- Given a set of data points $[X, y]$ we can optimize the parameters m on the cost function $J(m, y)$ with the following loop:

while not converged:

$$m = m - \alpha \nabla J(m, y)$$

- ∇J is the gradient of the cost function
- α is a hyper-parameter called the “learning rate”
- m is the set of parameters we want to optimize. This is the thing we learn.

```
next_x = 6  # We start the search at x=6
gamma = 0.01  # Step size multiplier
precision = 0.00001  # Desired precision of result
max_iters = 10000  # Maximum number of iterations
```

```
# Derivative function
```

```
def df(x):
    return 4 * x ** 3 - 9 * x ** 2
```

```
for _i in range(max_iters):
    current_x = next_x
    next_x = current_x - gamma * df(current_x)
```

```
    step = next_x - current_x
    if abs(step) <= precision:
        break
```

```
print("Minimum at ", next_x)
```

```
# The output for the above will be something like
# "Minimum at 2.2499646074278457"
```

- Taken from wikipedia
- gamma is the learning rate (alpha in previous slide)
- x is a single parameter (or theta).
- You can generalize this easily to multiple parameters with linear algebra.

Gradient of MSE

- θ is a vector of parameters (for an n-dimensional function)
- The cost function is MSE: $J(\theta) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x_i) - y_i)^2$
- The function we want to find parameters for is:
 - $f(x) = m_1 x_1 + c$
- The partial derivative of our cost function is:
 - $\frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i) x_i$
- Our gradient update is: $\theta_j = \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x_i) - y_i) x_i$

Conclusion

- By subtracting the gradient of the cost function from the m -parameters we optimize m
- This is known as Gradient Descent
- We learned the formal notation for gradient descent and a code version

Place your
Webcam Video here
Size 100%



Gradient Descent III

Prof. Gilad Gressel
Amrita School of Engineering

Introduction

- Linear algebra solutions
- Gradient Descent Parameters
 - Learning rate
 - Batch sizes
- Cost functions

Place your
Webcam Video here
Size 100%

Linear Algebra Solutions to Regression

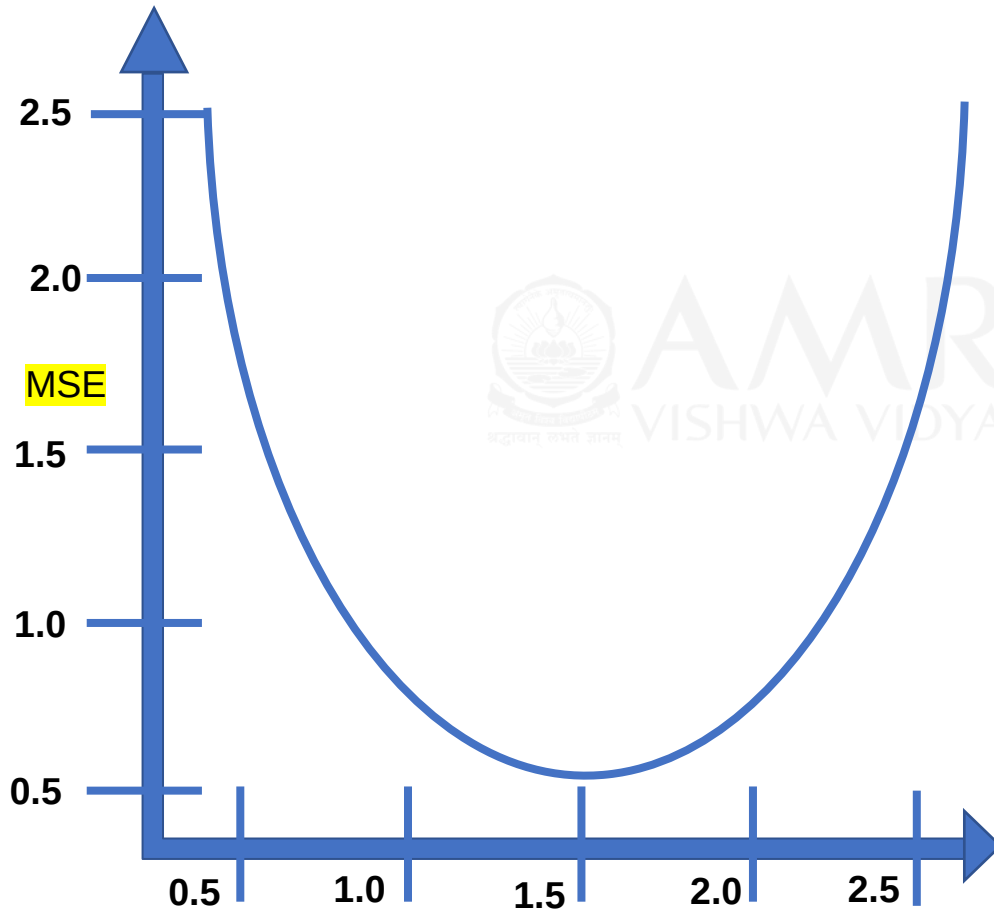
- You can solve this problem with a linear algebra equation that looks like this: $\theta = (X^T X)^{-1} X^T \vec{y}$
- Inverting the matrix becomes very expensive as the size of the matrix grows
- When the datasets / parameters get large enough, it's better to use gradient descent instead

Gradient Descent Parameters

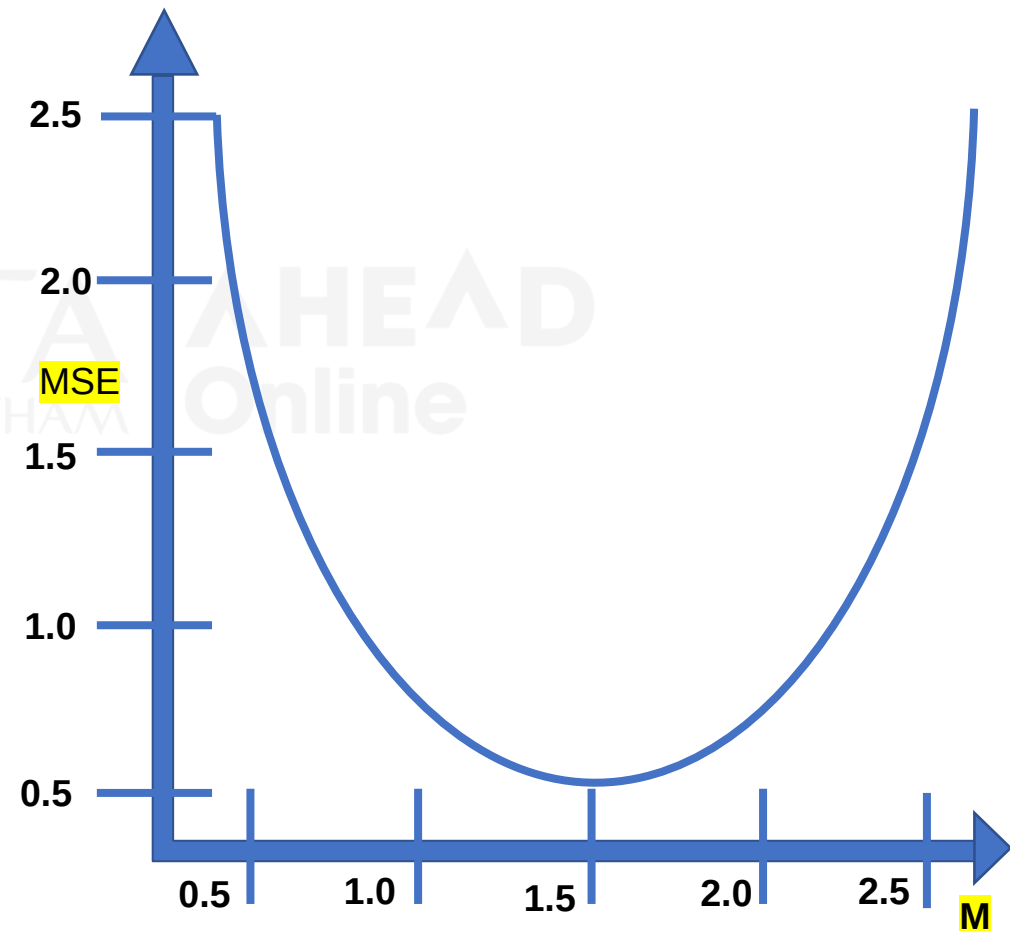
- ∇J is the gradient of the cost function
- α is a hyper-parameter called the “learning rate”
- m is the set of parameters we want to optimize. This is the thing we learn
- *Batch-size*: the amount of data that we want to optimize in one go

The Learning Rate

- Too Small



- Too large



Batch Sizes

- $\frac{1}{2n} \sum_{i=1}^n (y_i - f(x_i))^2$
- The n variable above, refers to **size** of the dataset.
- If the dataset is large, this becomes expensive quickly.
- It turns out you can do a variant of gradient descent called
 - Stochastic Gradient Descent
 - one sample at a time
 - Batch Gradient Descent
 - Chunks at a time (often 32, 64, 128, groups).
 - This gets very similar performance at **much** lower computational costs

Cost Functions other names

- Cost Function
- Loss Function
 - common in deep learning
- Objective Function



Cost Functions $\neq$ Scoring Metrics

- Cost functions are used in *optimization*
- Scoring metrics are used in *evaluation*
- Cost / loss / objective functions $\neq$ scoring function (error metric)
- *Sometimes* an error metric can *double* as both a scoring function and cost function (MSE)
- Cost / loss / objective functions **must be differentiable**
- Many scoring functions are not

Conclusion

- Using the gradient of the cost function, we can optimize the parameters of *any* function.
- Gradient descent is an optimization algorithm that is **widely** used in machine learning.
 - It's the heartbeat of deep learning.

Place your
Webcam Video here
Size 100%



Regularization

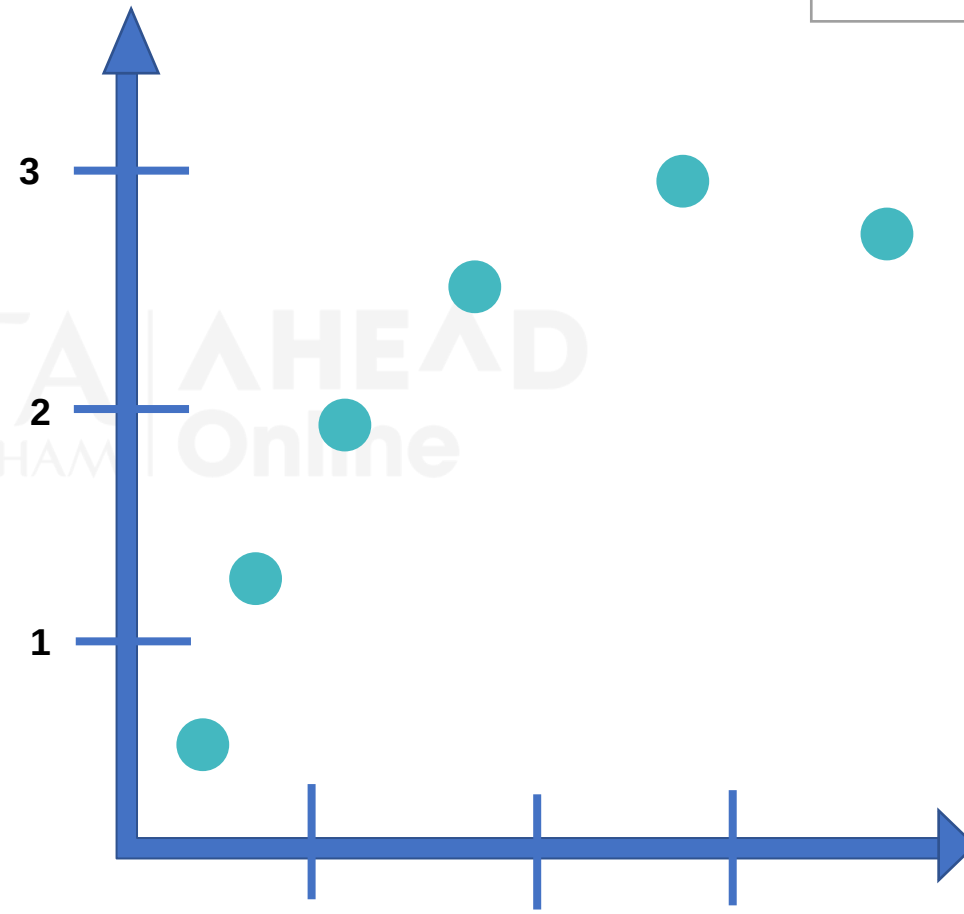
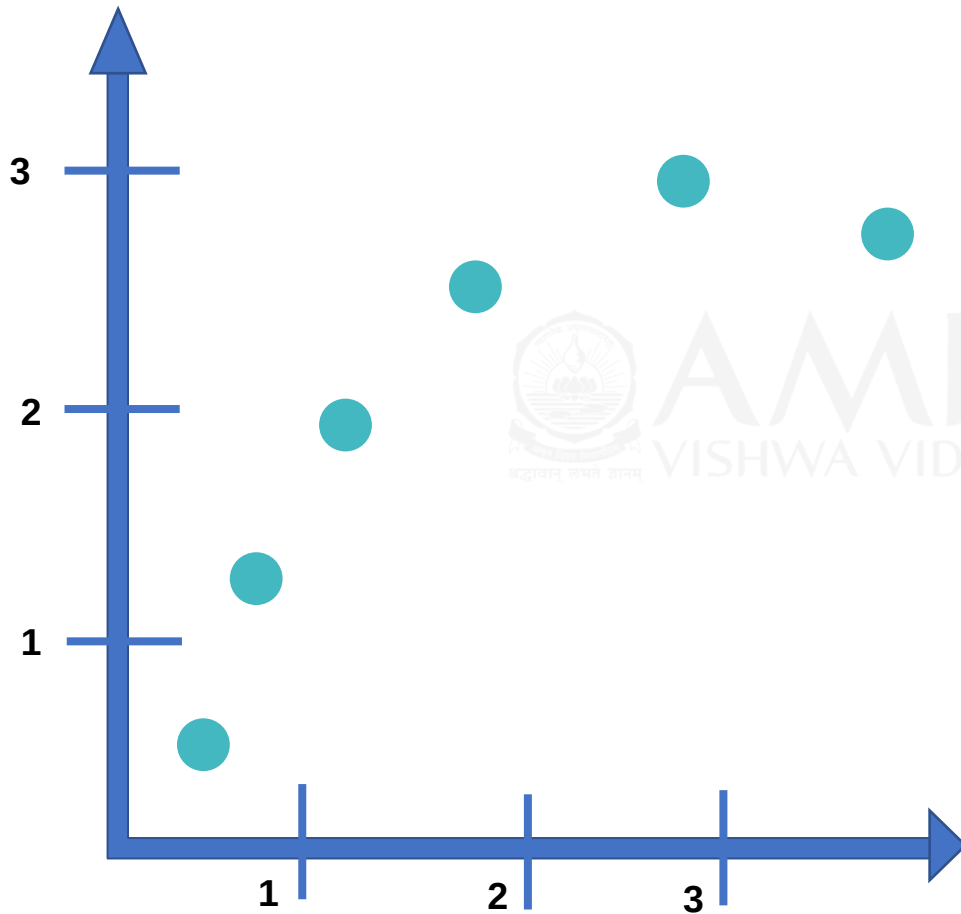
Prof. Gilad Gressel
Amrita School of Engineering

Introduction

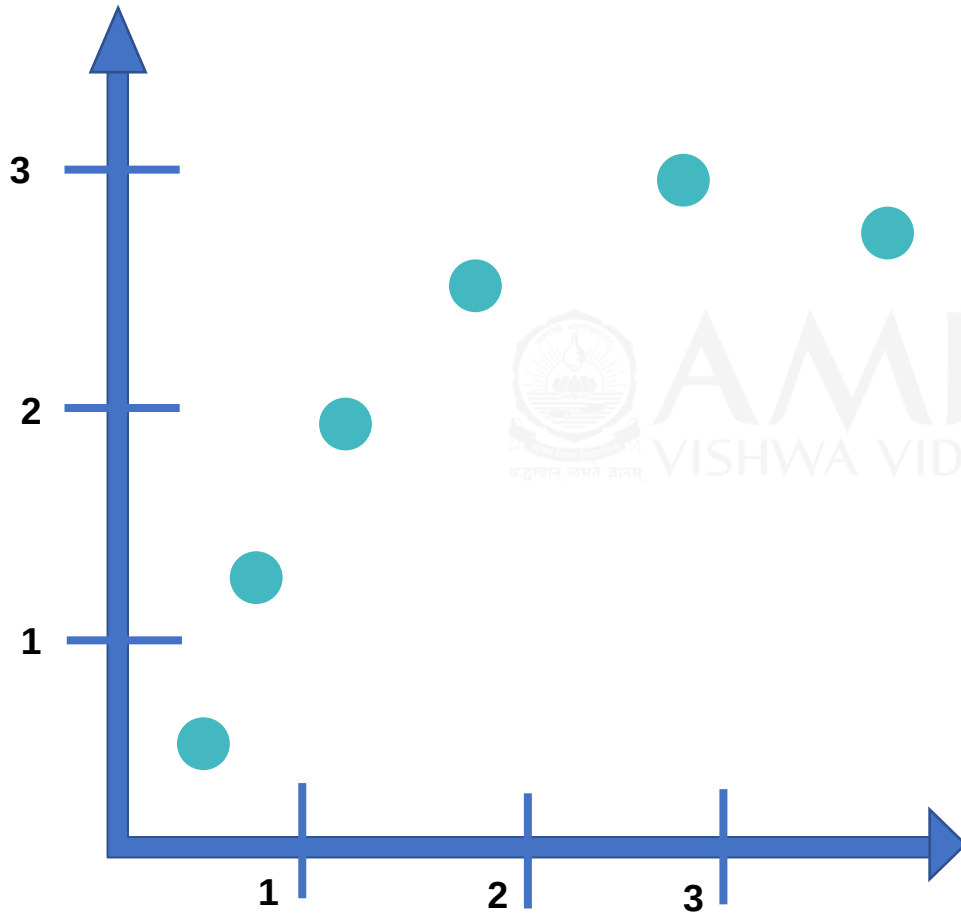
- How can we control overfitting within the algorithm?
- Regularization to the rescue
- Tuning it

Place your
Webcam Video here
Size 100%

Overfitting vs Underfitting

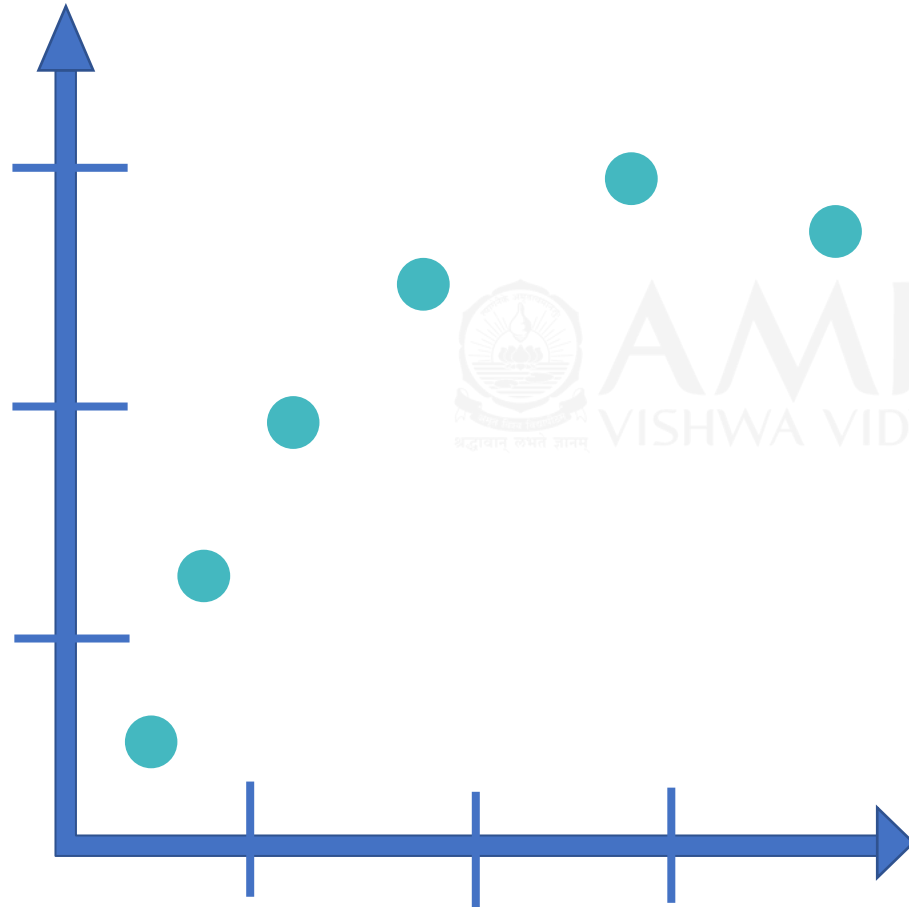


Regularization

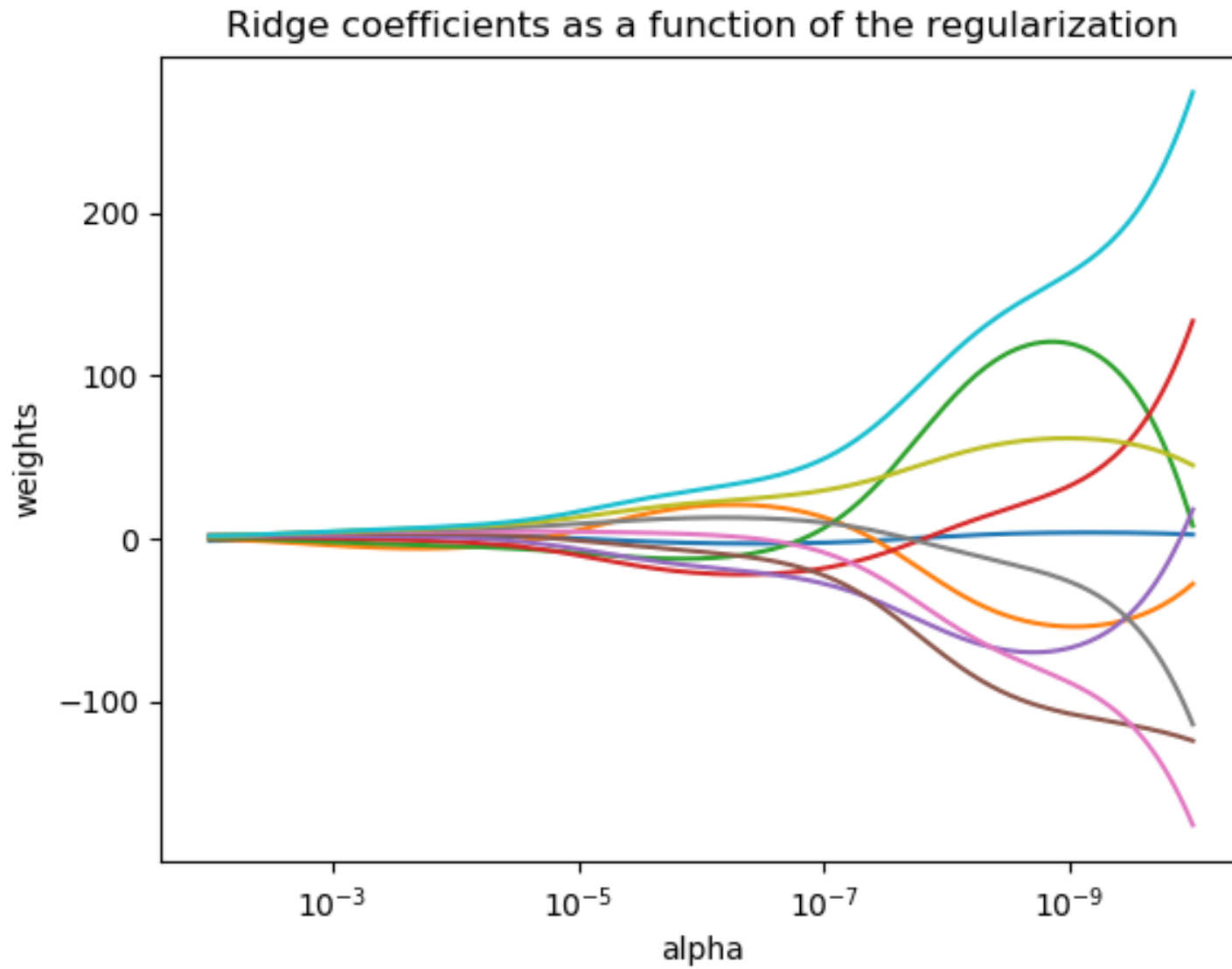


- In order to control against overfitting with linear / polynomial regressions, we can use a "regularization term"
- $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \alpha \sum_{j=1}^n \theta_j^2$
- In scikit learn's page they write it like this:
- $\min_w ||Xw - y||_2^2 + \alpha ||w||_2^2$

Regularization as a knob



- Think of it as a knob that you can twist
- More of the term means a smoother line
- Less of the term means a more complex line
- Our job is to figure out “how much” we need



https://scikit-learn.org/stable/auto_examples/linear_model/plot_ridge_path.html

Regularization as Jargon

- I believe that the term “regularization” originally we mathematical
- Today it's used as *any* technique which can be used to control overfitting



What if you can't Visualize the Data



- Let's assume we have 13 features, and our model is in 13 dimensions.
 - How do we figure out if we are overfitting? Underfitting?
- We will eventually answer this, but for now, just ponder.



Conclusion

- Regularization is a term that generally means “controlling” overfitting
- We looked at a mathematical formulation
- We pondered how to find overfitting without visual aids

Place your
Webcam Video here
Size 100%